CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

# EXPERIMENT – 4

| | |
|---|---|
| **NAME: Probal Dhali** | **UID: 24BAI71013** |
| **BRANCH:** CSE-AIML (CS221) | **SECTION/GROUP:** 24AIT_NTPP3[B] |
| **SEMESTER:** 5TH | **DATE OF PERFORMANCE:** 30/07/2026 |
| **SUBJECT:** Full Stack - II | **SUBJECT CODE:** 24CSP-337 |

## EXPERIMENT 4.1

### 1. AIM

To design and implement an interactive calendar interface for scheduling and managing posts.

### 2. OBJECTIVE

1. To understand time-based data visualization in UI systems
2. To implement calendar-based scheduling interfaces
3. To map structured data to temporal layouts
4. To enable user interactions such as drag-and-drop

### 3. SOFTWARE REQUIREMENT

- React.js
- Calendar library (e.g., FullCalendar / custom)
- Date utility libraries (optional)
- Code Editor

### 4. THEORY

Calendars are essential UI components for representing time-based data. In applications such as social media schedulers, events (posts) must be mapped to specific dates and time slots. A calendar system involves:

- **Temporal data modeling** (date, time, duration)
- **Event mapping** (linking posts to time slots)
- **User interaction handling** (click, drag, resize)

Interactive calendars enhance usability by allowing users to:

- Visually organize content
- Modify schedules dynamically
- Manage large datasets efficiently

**Academic Insight:**

Calendar systems fall under **Human-Computer Interaction (HCI)** and **Information Visualization**, where data is presented in a form that supports intuitive understanding and interaction.

### 5. IMPLEMENTATION/PROCEDURE

1. Design calendar UI layout (day/week/month view)
2. Map post data to calendar events
3. Render events dynamically
4. Implement click interactions (view/edit)
5. Add drag-and-drop scheduling *(optional/advanced)*
6. Sync calendar with application state

### 6. CODE

**Calendar**

```
import CalendarDay from "./CalendarDay";
```

```
export default function Calendar({
  calendarData,
  currentDate,
  getEventsForDate,
  onAdd,
  onEdit,
  onDelete,
  onDrop,
}) {
  const weekDays = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"];
  return (
    <div className="calendar">
      {/* Calendar Header */}
      <div className="calendar-header">
        {weekDays.map((day) => (
          <div key={day} className="week-day">
            {day}
          </div>
        ))}
      </div>
      {/* Calendar Body */}
      <div className="calendar-grid">
        {calendarData.map(({ date, isCurrentMonth }) => {
          const dateStr = date.toISOString().split("T")[0];
          const events = getEventsForDate(dateStr);

          return (
            <CalendarDay
              key={dateStr}
              date={date}
              isCurrentMonth={isCurrentMonth}
              events={events}
              onAdd={onAdd}
              onEdit={onEdit}
              onDelete={onDelete}
              onDrop={onDrop}
            />
          );
        })}
      </div>
    </div>
  );
}
```

**Post Service**

```
let posts = [
  { id: 1, title: "Do FS Experiment", description: "Complete the experiment", date: "2026-08-04", time: "08:00" },
  { id: 2, title: "UID - 24BAI71013", description: "Mark attendance", date: "2026-08-04", time: "12:30" },
  { id: 3, title: "24BAI71013@cuchd.in", description: "Email the Student", date: "2026-08-05", time: "18:00" },
```

```
  { id: 4, title: "Full Stack Development", description: "Happy Coding!", date: "2026-08-10", time: "09:00" }
];

let id = 5;

export const getPosts = () => [...posts];

export const addPost = post => posts.push({ ...post, id: id++ });

export const updatePost = (id, data) =>
  (posts = posts.map(p => p.id === id ? { ...p, ...data } : p));

export const deletePost = id =>
  (posts = posts.filter(p => p.id !== id));

export const movePostToDate = (id, date) =>
  (posts = posts.map(p => p.id === id ? { ...p, date } : p));
```

## 7. OUTPUT

The interactive calendar scheduling system was successfully implemented using React and the FullCalendar library. Posts were displayed as calendar events mapped to their respective dates, allowing users to visualize scheduled content in a monthly calendar view. The application supported event interactions such as viewing event details on click and rescheduling posts through drag-and-drop functionality, providing an efficient interface for content planning and schedule management.

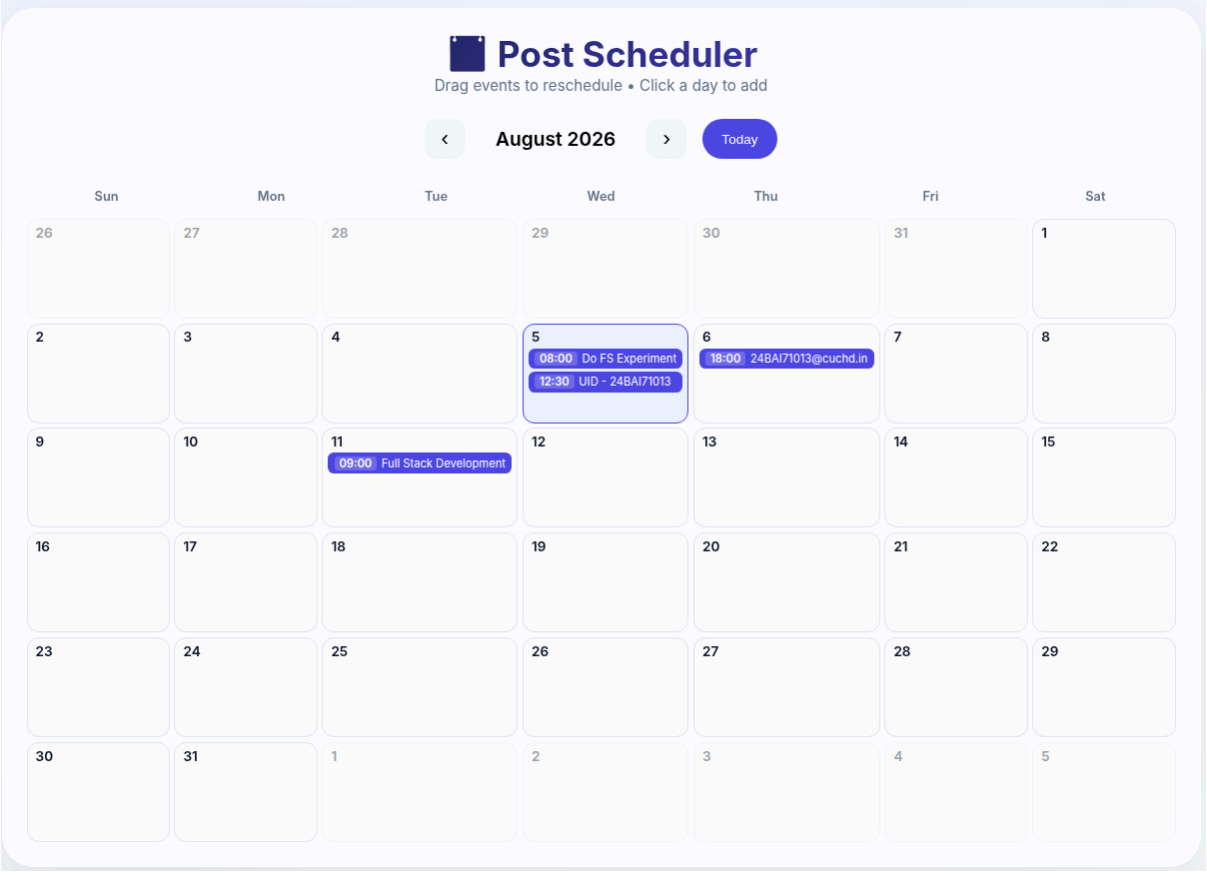


**Fig 4.1 Interactive calendar scheduling system**

## 8. LEARNING OUTCOMES

- Understand the concept of calendar-based scheduling in web applications.
- Design and implement an interactive calendar interface using React.
- Map structured post data to calendar events and time slots.
- Implement event handling for viewing and managing scheduled posts.
- Apply drag-and-drop functionality for dynamic event rescheduling.
- Synchronize calendar events with application state for efficient data management.
- Develop user-friendly scheduling systems for content planning and time-based applications.

CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

## EXPERIMENT 4.2

### 1. AIM

To optimize rendering performance and implement testing strategies for interactive UI components.

### 2. OBJECTIVE

1. To understand performance bottlenecks in UI systems.
2. To optimize rendering using memoization techniques.
3. To reduce unnecessary re-renders in React applications.
4. To implement testing for UI components and application logic.

### 3. SOFTWARE REQUIREMENT

1. React.js, Jest, Node.js & npm
2. React Testing Library
3. Browser DevTools
4. Visual Studio Code

### 4. THEORY

Interactive React applications such as calendars frequently update their UI based on user interactions. Without optimization, repeated rendering of components can negatively affect application performance and responsiveness.

Performance optimization techniques such as *React.memo, **useMemo, and **useCallback* help reduce unnecessary component re-renders and improve rendering efficiency. Memoization stores previously computed values and function references, ensuring computations occur only when required.

Testing is an essential part of frontend development. Unit testing verifies that UI components and application logic behave as expected under different scenarios, improving software reliability, maintainability, and stability.

By combining rendering optimization and testing, developers can build scalable, responsive, and reliable React applications.

### 5. IMPLEMENTATION/PROCEDURE

1. Create a React application and install the required dependencies.
2. Develop reusable React components for the calendar interface.
3. Identify components that re-render frequently.
4. Apply React.memo() to prevent unnecessary component re-rendering.
5. Use useMemo() to memoize expensive computations.
6. Use useCallback() to memoize event handler functions.
7. Optimize state updates to improve rendering performance.
8. Install Jest and React Testing Library.
9. Write unit test cases for calendar components.
10. Test user interactions such as event selection, drag-and-drop, and updates.
11. Execute test cases and verify successful execution.
12. Run the application and observe improved rendering performance.

### 6. CODE

**Calender**

```
import React, { useState } from "react";
import CalendarHeader from "./CalendarHeader";
import CalendarGrid from "./CalendarGrid";
```

```
import SearchBar from "./SearchBar";

export default function Calendar({ events }) {
  const [search, setSearch] = useState("");
  const [selected, setSelected] = useState(null);

  const filtered = events.filter(e =>
    e.title.toLowerCase().includes(search.toLowerCase())
  );
  return (
    <div className="calendar-container">
      <CalendarHeader />
      <SearchBar search={search} setSearch={setSearch} />
      <CalendarGrid
        events={filtered}
        onSelect={setSelected}
      />
      {selected && (
        <div className="event-details">
          <h2>📌 Event Details</h2>
          <h3>{selected.title}</h3>
          <p><b>Date:</b> {selected.date}</p>
          <p>{selected.description}</p>
        </div>
      )}
    </div>
  );
}
```

**Event**

```
const events = [
  { id: 1, title: "Probal Dhali", date: 5, color: "#3b82f6", description: "This is my name." },
   { id: 2, title: "Full Stack Development Workshop", date: 10, color: "#22c55e", description: "Learn full stack development skills." },
  { id: 3, title: "Experiment Review", date: 15, color: "#f97316", description: "Student Review." },
  { id: 4, title: "Placement Training", date: 21, color: "#ef4444", description: "Placement Preparation." },
  { id: 5, title: "24BAI71013", date: 28, color: "#8b5cf6", description: "University Identity Document." }
];

export default events;
```

**Files Used**

- App.js
- Calendar.js
- Calendar.test.js
- App.css

## 7. OUTPUT

The calendar application was successfully optimized using React.memo, useMemo, and useCallback, resulting in reduced unnecessary component re-renders and improved rendering performance. Unit tests executed successfully using Jest and React Testing Library, verifying the correctness of UI components and user interactions. The optimized application provided a faster, more responsive, and reliable user experience.

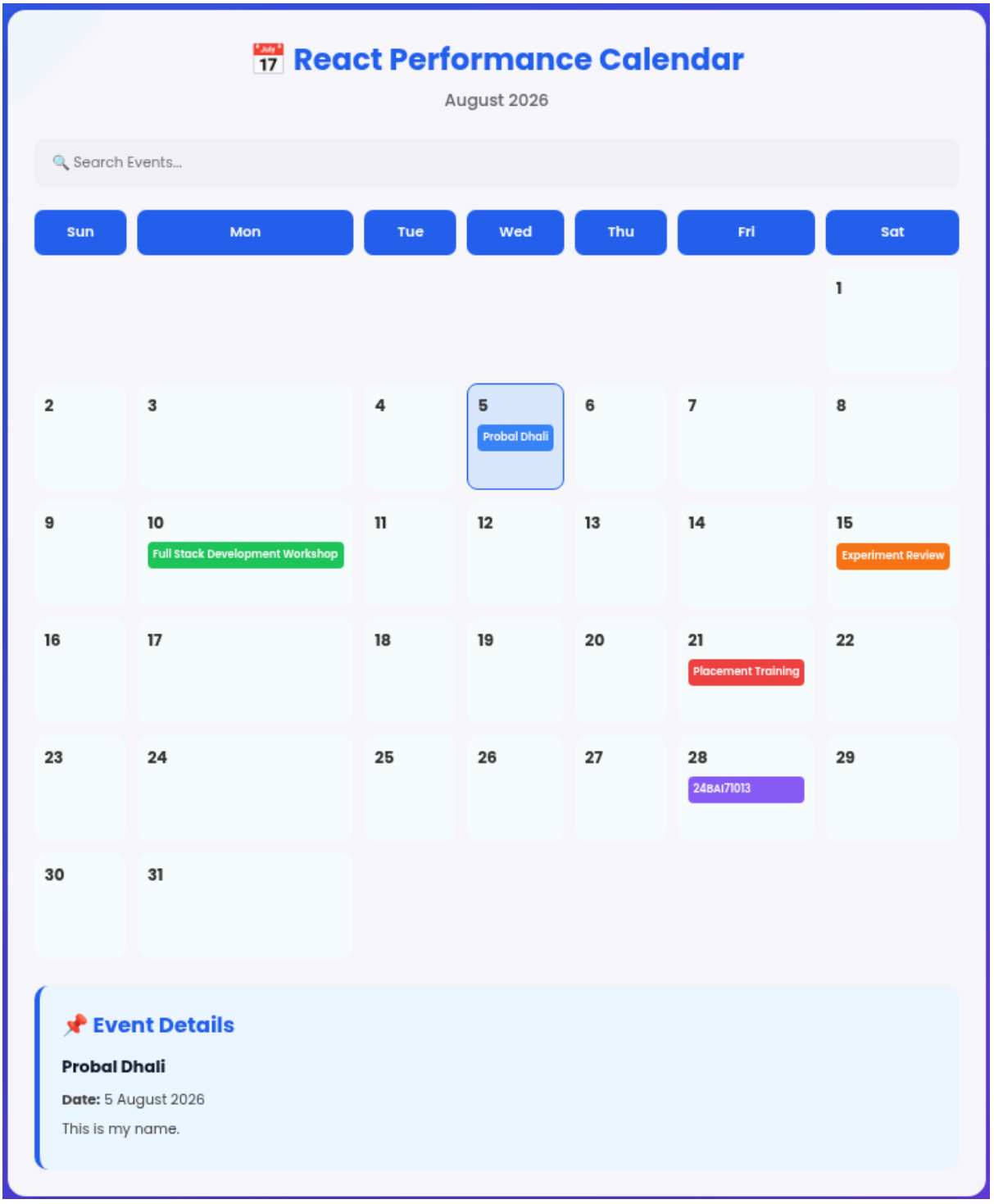


**Fig 4.2: Calendar Application**

**8. LEARNING OUTCOMES**

- Understand rendering performance optimization techniques in React.
- Implement memoization using *React.memo, **useMemo, and **useCallback*.
- Reduce unnecessary component re-renders to improve application efficiency.
- Optimize state updates for interactive UI components.
- Develop and execute unit tests using Jest and React Testing Library.
- Validate component functionality and user interactions through testing.
- Build scalable, optimized, and maintainable React applications following performance engineering and software testing principles.